

TPO Ajedrez — Roadmap de ejecución

Versión 2: contrato del núcleo corregido y plan fase por fase

Fase	Fechas	Qué se logra
F0 · Setup y contratos	19/09 - 27/09	Repo, build, CI e interfaces acordadas y commiteadas
F1 · Walking skeleton	28/09 - 04/10	Una jugada real atravesando todas las capas
F2 · Piezas y captura	05/10 - 18/10	Las seis piezas con su movimiento propio
F3 · Turnos, validación y jaque	19/10 - 25/10	Hito MVP: alcance obligatorio completo
F4 · GUI y extensibilidad	26/10 - 01/11	Swing jugable + pruebas de pieza nueva y 10x10
F5 · Opcionales	02/11 - 12/11	Funcionalidades extra; congelamiento el 12/11
F6 · Cierre y defensa	13/11 - 20/11	Entregables y simulacro individual

Qué cambió respecto de la versión 1. Se sacaron las abstracciones que no resolvían ningún problema real y que además introducían un error de diseño. El detalle está en la sección 1; el contrato ya corregido, en la sección 2. Si ya leíste el plan de acción, esas dos secciones son lo único nuevo: el resto es el paso a paso para ejecutar.

1. Correcciones al diseño original

El criterio de la revisión fue simple: toda abstracción tiene que resolver un problema concreto de hoy o una extensión probable de la defensa. Una interfaz con una sola implementación y ningún consumidor polimórfico no es diseño, es decoración, y en la defensa juega en contra: el enunciado evalúa el *por qué* de cada decisión.

Elemento de la v1	Veredicto	Qué queda en la v2
BoardView	Forzada, y además rompía la simulación de jaque: las reglas recibían BoardView pero Move.applyTo exige un Board.	Eliminada. Todo trabaja sobre Board, que al ser inmutable ya impide que alguien lo modifique.
BoardSetup como interfaz	Nadie la consumía de forma polimórfica.	Clases sueltas: StandardSetup y, si la piden, TenByTenSetup.
StandardPieceTypes inyectado	Inyección de adorno: clase concreta, sin estado ni alternativa posible.	Métodos estáticos. StandardSetup sin parámetros.
Game(BoardSetup, ...)	Indirección sin motivo: el juego necesita un tablero, no una fábrica.	Game(Board, MoveValidator, CheckDetector). Main arma el tablero.
CheckDetector como interfaz	Una sola implementación; simular el jaque con un doble en un test de ajedrez confunde más de lo que ayuda.	Clase concreta, igual inyectada por constructor (sigue siendo inyección de dependencias).
GameEventListener (Observer)	Forzado: consola y GUI llaman a play() y reciben la respuesta en el acto. Nadie necesita que le avisen. onInvalidMove duplicaba MoveResult.	Eliminado. MoveResult informa el rechazo y el jaque. Vuelve si agregan una IA, donde sí hay jugadas que la interfaz no inició.
BoardRenderer como interfaz	Strategy con una sola estrategia.	Clase concreta AsciiBoardRenderer hasta que exista un segundo renderer.
DestinationNotOwnPieceRule	Parcheaba un problema de las reglas de movimiento, que generaban jugadas sobre piezas propias.	Las reglas de movimiento ya no las generan. El validador queda con dos reglas.
Etiquetas de patrones	Sobreventa: llamar Chain of Responsibility a una lista de reglas y Command a Move.applyTo invita a que el profesor pregunte dónde está el patrón.	Se nombran como lo que son: lista de reglas de validación, y la jugada como objeto.

Lo que no se toca, porque no estaba forzado: piezas por composición de MovementRule, Board inmutable, la lista de MoveValidationRule (es el punto genuino de extensión sin modificar), el puerto ChessGame, Move como interfaz (hace que enroque, promoción y captura al paso sean clases nuevas en vez de modificaciones) y los módulos Maven.

Para el documento de justificación, cada línea de esta tabla es una entrada. Decir "no usamos Observer porque ninguna parte del sistema necesita enterarse de algo que no pidió, y lo agregaríamos con un oponente de IA" vale más que defender un patrón vacío.

2. El contrato del núcleo (se congela en la Fase 0)

Esto es lo que se acuerda entre los cinco y se commitea antes de que nadie implemente nada, con los métodos vacíos o lanzando `UnsupportedOperationException`. A partir de ese commit, cada uno puede avanzar en su paquete sin pisar a los demás.

model/	Position(file, rank) Direction(dx, dy) Color Board Piece PieceType	record, con offset(Direction) record + constantes ORTHOGONAL, DIAGONAL, ALL, KNIGHT_JUMPS enum WHITE/BLACK, opposite(), forward() IMMUTABLE: width, height, isInside, pieceAt, findRoyal(Color), with(pos, piece), without(pos) color, type, hasMoved, candidateMoves(Board, from) name, royal, List<MovementRule>
movement/	MovementRule SlidingRule(dirs, maxSteps) PawnAdvanceRule Move StandardMove	interfaz: candidateMoves(Board, Position, Piece) : List<Move> LeapRule(offsets) PawnCaptureRule interfaz: from(), to(), applyTo(Board) : Board
setup/	StandardPieceTypes StandardSetup	estático: pawn(), rook(), knight(), bishop(), queen(), king() createBoard() : Board
rules/	CheckDetector MoveValidationRule OwnPieceRule MoveValidator(List<MoveValidationRule>)	clase: isInCheck(Board, Color) interfaz: check(Board, Move, Color turn) : ValidationResult DoesNotLeaveKingInCheckRule(CheckDetector)
game/	ChessGame Game(Board, MoveValidator, CheckDetector) MoveResult	interfaz: play(from, to), board(), currentTurn(), isInCheck(Color) ok / reason / checkedColor

Regla que no se negocia: las reglas de movimiento NO generan jugadas sobre piezas propias. Así las jugadas candidatas ya salen limpias, el validador solo se ocupa del turno y del auto-jaque, y más adelante la GUI puede resaltar jugadas legales sin filtrar nada.

Dónde queda cada concepto de la cursada, en una línea

Concepto	Dónde
Núcleo vs adaptador	chess-core vs chess-console y chess-gui-swing
Puerto de entrada	ChessGame (no hay puerto de salida: el núcleo no necesita nada del exterior)
Inyección por constructor	Game, MoveValidator, DoesNotLeaveKingInCheckRule, PieceType, SlidingRule, ConsoleAdapter
Raíz de composición	Main: único lugar con new de dependencias
Composición sobre herencia	Las piezas son listas de MovementRule: reina = torre + alfil
Strategy	MovementRule
Factory	StandardPieceTypes, StandardSetup
Abierto/cerrado	Pieza nueva, regla de validación nueva o interfaz nueva = clases nuevas, sin tocar lo existente
Inmutabilidad	Board, Position, Move: permite simular el auto-jaque sin efectos secundarios
Dobles de prueba	Tableros armados a mano en los tests; Scanner sobre String para probar la consola

3.1 Modelo, movimiento y setup (v2)

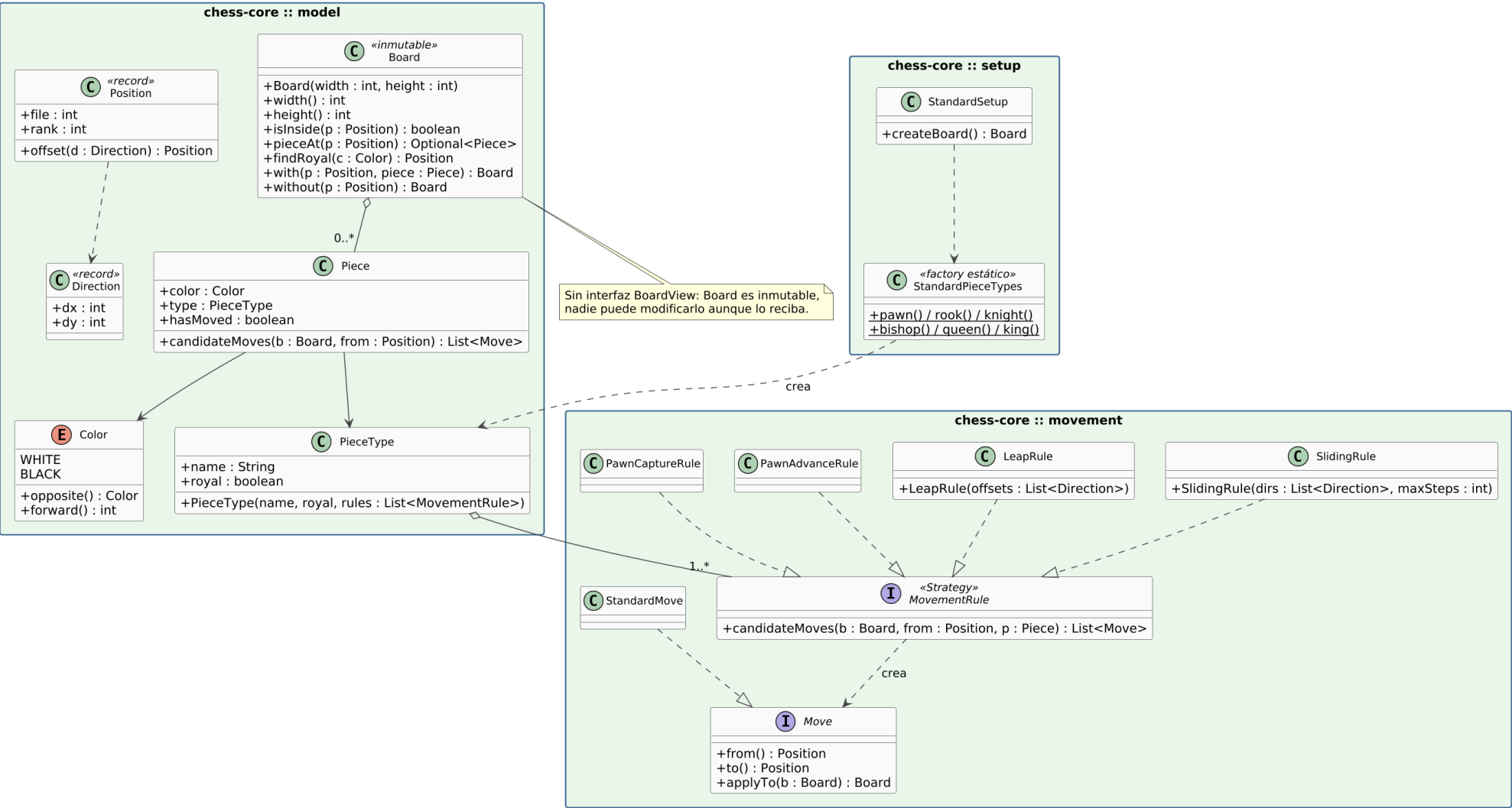


Figura 1. Las piezas no heredan unas de otras: cada PieceType se compone de una o más MovementRule.

3.2 Reglas y juego (v2)

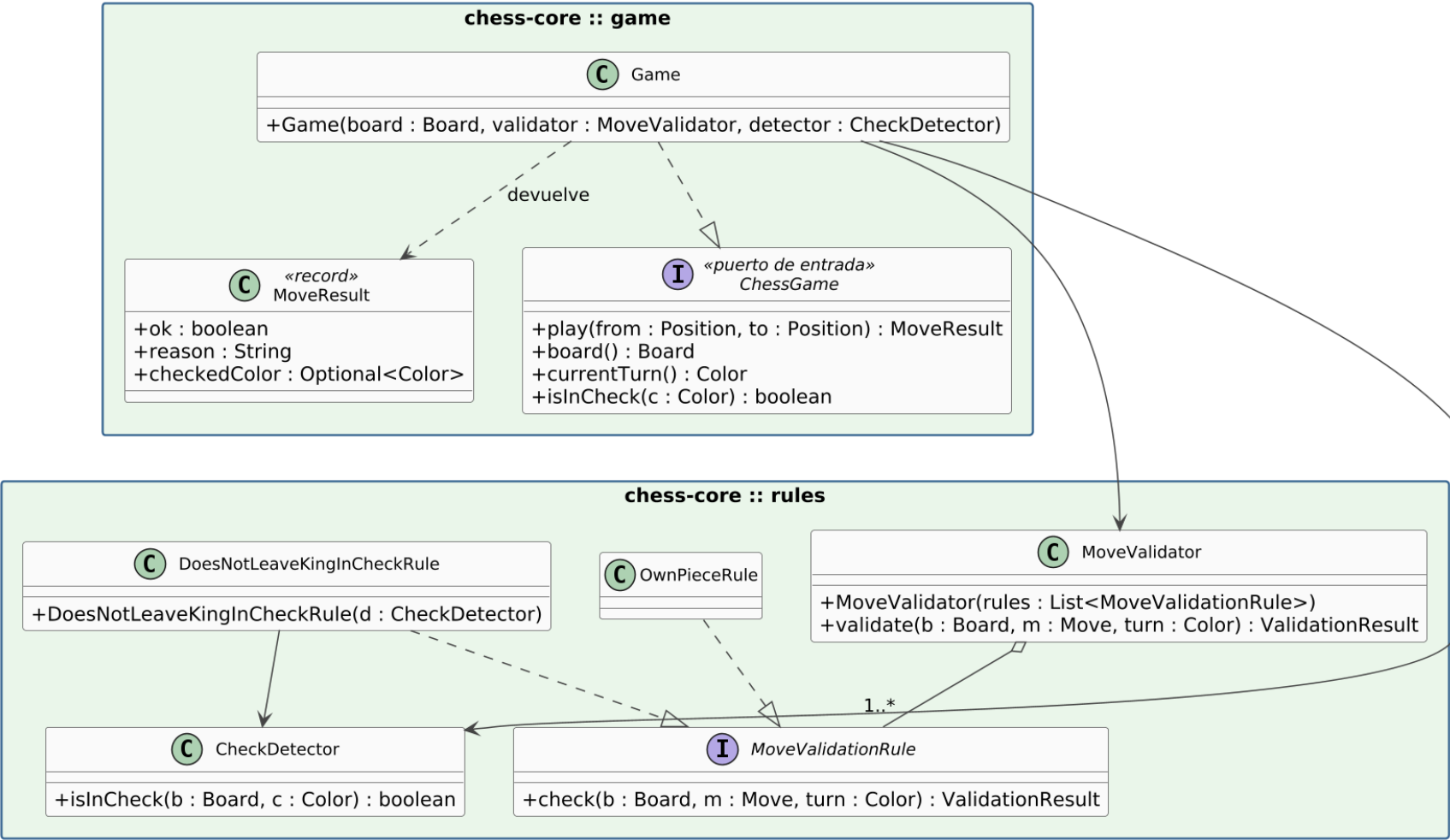
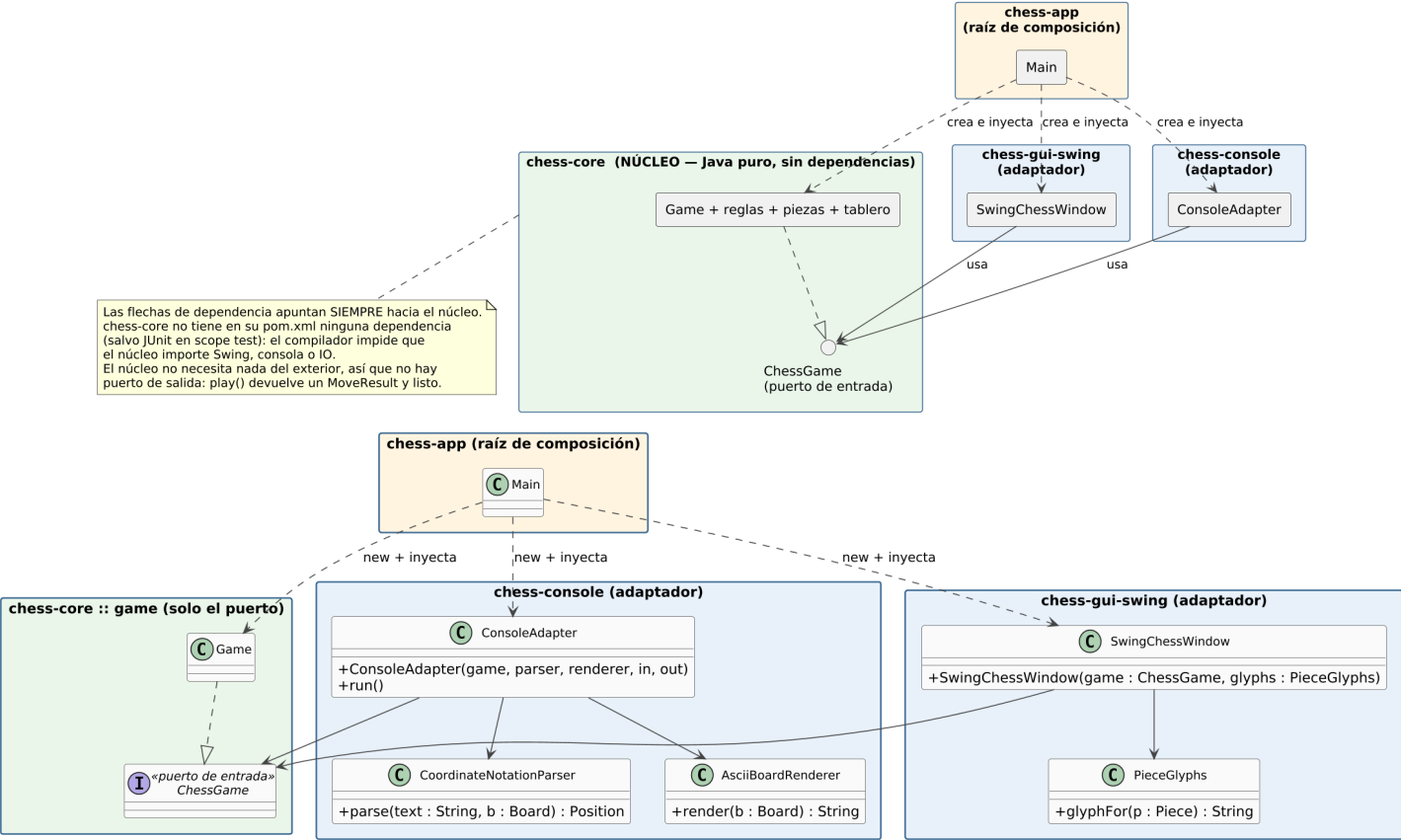


Figura 2. Game recibe tablero, validador y detector; el validador recibe su lista de reglas.

3.3 Módulos, adaptadores y raíz de composición (v2)



Figuras 3 y 4. Todas las dependencias apuntan al núcleo; Main es quien crea y conecta.

4. Roadmap fase por fase

F0 · Setup y contratos (19/09 - 27/09)

Es la única fase donde conviene trabajar los cinco juntos, no repartidos.

Tareas	1) Repo en GitHub con .gitignore de Java y Maven. 2) pom.xml padre con Java 21, JUnit 5 y los cuatro módulos: chess-core, chess-console, chess-gui-swing, chess-app. El núcleo sin ninguna dependencia fuera de JUnit en scope test. 3) .github/workflows/ci.yml corriendo mvn -B test en cada push. 4) Reunión de contratos: crear y commitear las interfaces y records de la sección 2, con los métodos vacíos. 5) CLAUDE.md en la raíz con las reglas del proyecto (ver sección 5). 6) docs/uml/ con los .puml y docs/decisiones.md con la plantilla qué / por qué / cuándo romperlo.
Terminado cuando	Cualquiera clona, corre mvn test y da verde. El CI también. El núcleo no tiene dependencias.

F1 · Walking skeleton (28/09 - 04/10)

Una sola jugada atravesando todas las capas. Es la fase que más ordena: no la saltéen.

Tareas	1) Position, Direction, Color y Board con with / without. 2) Piece, PieceType, StandardMove. 3) PawnAdvanceRule solo con el avance de una casilla. 4) Game.play con un MoveValidator de lista vacía. 5) CoordinateNotationParser, AsciiBoardRenderer, ConsoleAdapter y Main. 6) Un setup mínimo con unos pocos peones.
Primeros tests	pawn_advancesOneSquare, pawn_cannotMoveBackwards, board_isInside_respectsDimensions.
Terminado cuando	Corrés Main, escribís e2 e3 y el tablero se redibuja con el peón movido.

F2 · Las seis piezas y la captura (05/10 - 18/10)

Acá rinde el TDD: primero los tests de la pieza, después la regla.

Tareas	1) SlidingRule y LeapRule con sus direcciones, respetando bloqueo y bordes, sin generar jugadas sobre piezas propias. 2) PawnAdvanceRule completa (doble paso con hasMoved, nunca "si está en la fila 2") y PawnCaptureRule. 3) StandardPieceTypes con las seis piezas: la reina es torre más alfil; el rey, un deslizamiento de un paso. 4) StandardSetup con la posición inicial completa. 5) Captura en StandardMove.applyTo.
Tests	Una clase por pieza: desde el centro con tablero vacío, contra el borde, bloqueada por pieza propia, capturando rival, casos inválidos.
Terminado cuando	Se puede jugar en consola moviendo cualquier pieza, todavía sin control de turno ni jaque. UML actualizado.

F3 · Turnos, validación y jaque (19/10 - 25/10) — Hito MVP

Tareas	1) Alternancia de turnos en Game. 2) OwnPieceRule y MoveValidator con la lista de reglas. 3) CheckDetector: recorre las jugadas candidatas del rival y ve si alguna cae sobre la pieza royal. 4) DoesNotLeaveKingInCheckRule: aplica la jugada (que devuelve un tablero nuevo) y le pregunta al detector. 5) MoveResult informando el motivo del rechazo y el jaque; la consola lo muestra.
Tests	Turnos; jaque por cada tipo de pieza atacante; pieza clavada; rey que no entra en casilla atacada; salir del jaque capturando o interponiendo.
Terminado cuando	Hito MVP: toda la matriz de trazabilidad del plan tiene evidencia en verde. Revisión grupal del código contra el enunciado.

F4 · GUI y prueba de extensibilidad (26/10 - 01/11)

Dos adaptadores sobre el mismo núcleo es la mejor demostración posible de la arquitectura en la exposición.

Tareas	1) SwingChessWindow: grilla de width x height botones, clic en origen y clic en destino, repintado tras la jugada. 2) PieceGlyphs: nombre de pieza a glifo Unicode, con la inicial como fallback. Esto es lo que permite que una pieza nueva se vea sin tocar la GUI. 3) Main --gui. 4) Los dos tests que valen oro para la defensa: una pieza compuesta nueva definida solo dentro del test , y una partida sobre un tablero de 10x10.
Terminado cuando	Se juega por consola y por GUI, y los tests de extensibilidad pasan sin haber modificado ninguna clase del núcleo.

F5 · Opcionales (02/11 - 12/11)

De a una funcionalidad, terminada con tests y UML antes de empezar la siguiente.

Tareas	Orden sugerido: 1) jaque mate y ahogado (reutiliza el validador para generar jugadas legales); 2) deshacer (casi gratis con el tablero inmutable); 3) promoción de peón; 4) resaltar jugadas legales en la GUI; 5) enroque; 6) IA con RandomPlayer o minimax.
Terminado cuando	El 12/11 se congela: lo que no esté terminado y en verde, no entra. Mejor dos opcionales impecables que cinco a medias.

F6 · Cierre y defensa (13/11 - 20/11)

Tareas	1) Pasada final del UML contra el código, clase por clase. 2) Documento de justificación a partir de decisiones.md, incluyendo las decisiones de no usar algo. 3) PPTX y ensayo con tiempo. 4) Simulacro: cada integrante, solo y con reloj, agrega una pieza nueva o cambia el tablero a 10x10.
Terminado cuando	Los cinco entregables están listos y los cinco integrantes pasaron el simulacro.

5. Reglas del proyecto (contenido del CLAUDE.md)

Pegar esto en la raíz del repo. Sirve tanto para el equipo como para que Claude Code respete el diseño en vez de improvisar el suyo.

```
# TP0 Ajedrez – reglas del proyecto

## Arquitectura
- chess-core es el núcleo: Java puro, sin dependencias (solo JUnit en scope test).
- Prohibido en el núcleo: System.out, Scanner, javax.swing, archivos, red.
- chess-console y chess-gui-swing son adaptadores: solo llaman a ChessGame y muestran resultados.
- Ninguna regla del ajedrez vive en un adaptador.

## Diseño
- Ninguna clase hace new de sus colaboradores: los recibe por constructor en campos final.
  La única excepción es Main (raíz de composición).
- Las piezas NO heredan entre sí: se componen de MovementRule.
- Prohibido switch o instanceof sobre tipos de pieza.
- Board es immutable: with/without devuelven un Board nuevo.
- Nada de literales 8: el tablero conoce su width y height.
- El doble paso del peón depende de hasMoved, nunca de un número de fila.
- No agregar una interfaz si va a tener una sola implementación y ningún consumidor polimórfico.

## Convenciones
- Clases, métodos y variables en inglés. Comentarios y documentación en español.
- Tests: JUnit 5, nombre sujeto_condicion_resultadoEsperado, bloques Arrange/Act/Assert comentados.
- Si cambia una firma pública del núcleo, se actualiza docs/uml/*.puml en el MISMO commit.
- Si se toma una decisión de diseño, se agrega una entrada a docs/decisiones.md.
```

6. Reparto y convivencia en main

Rol	Paquetes que le tocan
P1 • Modelo	Position, Direction, Color, Board, Piece, PieceType, Move, StandardMove, StandardSetup. Guardián del UML.
P2 • Movimiento	SlidingRule, LeapRule y torre, alfil, reina, rey y caballo. Test de pieza nueva en F4.
P3 • Peón y jaque	PawnAdvanceRule, PawnCaptureRule, CheckDetector, DoesNotLeaveKingInCheckRule. Test de 10x10 en F4.
P4 • Juego	Game, ChessGame, MoveResult, MoveValidator, OwnPieceRule, Main. Dueño de decisiones.md.
P5 • Adaptadores	CoordinateNotationParser, ConsoleAdapter, AsciiBoardRenderer, SwingChessWindow, PieceGlyphs. Dueño de la PPTX.

Regla de convivencia	Por qué
git pull --rebase antes de empezar y antes de cada push	Evita commits de merge y reduce conflictos.
Commits chicos, un tema por commit	Si algo rompe, se identifica y se revierte fácil.
Nunca pushear con tests en rojo	El que rompe main frena a los otros cuatro.
Cambiar una firma del contrato se avisa antes	Una firma que cambia rompe el trabajo de otros en silencio.
Revisión cruzada: cada uno entiende el código de otro rol	La defensa es individual y sobre todo el proyecto.

Definición de terminado de cualquier tarea: compila; mvn test en verde; tiene tests si es núcleo; si cambió una firma pública, el .puml se actualizó en el mismo commit; si hubo una decisión de diseño, quedó una entrada en decisiones.md.

Lo primero, hoy

Crear el repo y el pom.xml padre con los cuatro módulos, y en la misma sesión commitear las interfaces de la sección 2. Mientras no exista ese commit, el resto del grupo no puede avanzar sin riesgo de rehacer trabajo.